AI ACADEMY · LEVEL 6 · GRADE 11 · WEEK 22 · STUDENT LESSON

Queues and asynchronous work

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Grounded Learning Service

THE CONNECTED WEEK

A receipt starts the story; checked completion finishes it.

Trace job states and repeated delivery without inventing completed work or repeated effects.

Keep one habit: State contract → commit/ack ledger → J12 picture → uncertainty probes → sequential mock → independent three-job report.

MAIN LESSON · SECTION 1

Track a job from receipt to a checked result

> Count accepted work, completed work and deliveries separately.

Learning goals

I can explain the **mechanism** and terminology of **queue**s and asynchronous work.

I can apply the stated procedure to a new case with a traceable result.

I can identify an assumption or limitation and design a revealing follow-up.

Retrieve identifiers and committed-result recovery

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Caching and stale information. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Sandboxing and resource limits. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Timeouts retries and cancellation. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Prerequisite and starting response

- Recall Caching and stale information. Explain why a recorded result needs its **input** conditions and **evidence** source before it can support a claim.

> **My starting explanation**

End goal

Explain **queue**s and asynchronous work using the deciding step in this fictional service.

▼ **Earlier wording, alternatives and picture transcripts**

Your learning target

Remember before reading

This week you will practise:

Explain the **mechanism** and terminology of **queue**s and asynchronous work

Apply the stated procedure to a new case with a traceable result

Identify an assumption or limitation and design a revealing follow-up

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Distinguish a submission receipt from a worker acknowledgement

> The two signals occur at different points in the job lifecycle.

Begin with a library request receipt

Imagine handing a library card to a helper. A receipt means the request was accepted; it does not mean the book is ready. A queue works similarly: it holds jobs for a worker. Keep the same job ID when work is delivered again. Before repeating an effect, consult reliable saved completion evidence. A completed result can be returned again without doing the effect again; uncertain status requires reconciliation.

Define acceptance and repeated delivery

Queues separate acceptance from completion. Work may be redelivered, so consumers must preserve operation IDs and deduplicate committed effects.

Complete state and completion-evidence contract

Term or assumption	Exact lesson meaning
Accepted / pending	Receipt exists; work has not completed

Term or assumption	Exact lesson meaning
Running	Worker started; completion is not established
Completed	Declared checked-result and commit step recorded
Failed	Failure observed and recorded; retry needs an explicit policy
Worker acknowledgement	Signal to queue after declared durable completion; distinct from submission receipt
Repeated completed ID	Return prior result; acknowledge without repeating effect
Unknown or uncertain status	Review and reconcile; do not assume safe to repeat
Commit assumption	Reliable saved status must correspond to the effect; a real atomicity or provider-deduplication design is required
Model limit	Sequential in-memory fixtures and copied snapshots do not implement persistence, concurrency or exactly-once delivery

Name supplied and missing evidence

List the given facts and one fact that is not supplied.

▼ Earlier wording, alternatives and picture transcripts

Begin by writing the question the procedure is intended to answer. Identify the information already supplied, the information it will produce and the information still missing.

These are constructed classroom examples; they do not require live accounts, private records or an external service. Do not treat a plausible explanation as a measured outcome.

MAIN LESSON · SECTION 3

Trace two deliveries with one committed mock effect

> A crash before acknowledgement does not erase the supplied completion evidence.

Original durable-completion rule

A **queue**d job is pending, running, completed or failed. Acknowledge only after the declared durable completion step. A repeated completed ID returns its prior result; unknown status needs reconciliation.

Complete event and effect-count ledger

Main job J event	Recorded state / queue signal	Mock effects so far
Submission accepted	Pending; no completed result	0
First worker delivery starts	Running; no acknowledgement yet	0
Declared checked result and commit recorded	Completed result saved	1
Crash before worker acknowledgement	Completed remains; queue may redeliver	1
Second delivery with same J and payload	Read committed result; acknowledge	1
Unknown ID or running-only record	Review; no new effect from recovery decision	Unchanged
Same ID with changed payload	Reject conflict; not a new legitimate retry	Unchanged

State which evidence supports recovery

Before applying the procedure, write its units, denominator or decision boundary where relevant. A partner should be able to repeat each step using the same **input**s.

If a required **input** is absent or violates the stated contract, record that condition explicitly instead of quietly replacing it with a convenient value.

Explain the visual

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

Queues and asynchronous work — evidence trace

INPUT AND SCOPE	CHECK OR CALCULATION	CONCLUSION AND LIMIT
Worker commits job J then crashes before queue acknowledgement. J is delivered again.	A queued job is pending, running, completed or failed. Acknowledge only after the declared durable completion step. A repeated completed ID returns its prior result; unknown status needs reconciliation.	Consult durable job status; return committed result and acknowledge without repeating the effect. Delivery count is not effect count.

The figure connects this chapter's worked input, declared procedure and result. Read the complete worked reasoning for qualifiers and intermediate steps; the abbreviated labels do not replace that reasoning.

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Predict the status after an observed pre-output crash

> A receipt cannot support a success claim.

Current task and response

- Worker commits job J then crashes before **queue acknowledgement**. J is delivered again. Show the intermediate steps and give the conclusion with its scope.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the changed crash case

- Change: The worker crashes before producing checked **output**.

▼ **Check after predicting**

Prediction: The job is failed or awaiting a controlled retry, not succeeded.

Reason: Receipt is not **evidence** that conversion finished.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Compare committed-result recovery with a pre-output crash

> The main crash happens after commit; the pictured crash happens before checked output.

Main crash-after-commit case and checked conclusion

Worker commits job J then crashes before **queue acknowledgement**. J is delivered again.

Consult durable job status; return committed result and acknowledge without repeating the effect. Delivery count is not effect count.

Compare the intermediate **evidence** with the final statement. Explain why each number or decision follows from the stated **input**s.

The conclusion belongs to this scenario and procedure; changing the **input**s, evaluation population or authority can change what it establishes. Retain the raw record so a later revision can be compared honestly.

Separate J12 receipt, state vocabulary and failure evidence

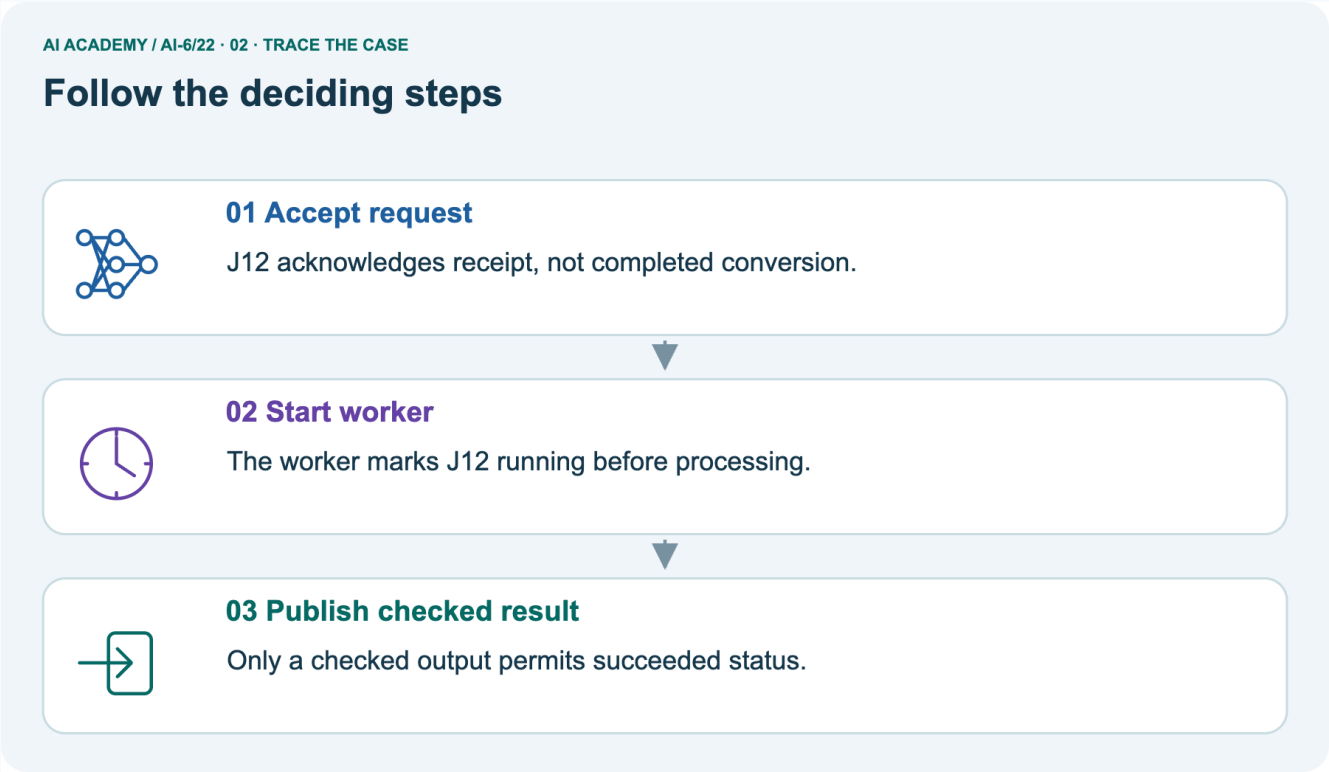
Queued is not complete. A conversion service **queue**s job J12 rather than **process**ing it during submission.

States: **queue**d, running, succeeded, failed. Submission returns J12 with **queue**d status.

The picture uses queued/running/succeeded/failed; the main example uses pending/running/completed/failed. Queued corresponds to pending and succeeded to completed for these classroom cases. J12's submission receipt acknowledges acceptance, not the worker's later delivery completion. Only checked output permits succeeded.

If a supervisor observes and records a crash before checked output, failed is justified; a declared retry could return it to queued. If nobody has yet observed the crash, the saved status may remain running and needs reconciliation. Do not invent a success, an observed failure or a retry policy from the mere absence of output.

Trace receipt, start and checked output



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Accept request	J12 acknowledges receipt, not completed conversion.
2. Start worker	The worker marks J12 running before processing.
3. Publish checked result	Only a checked output permits succeeded status.

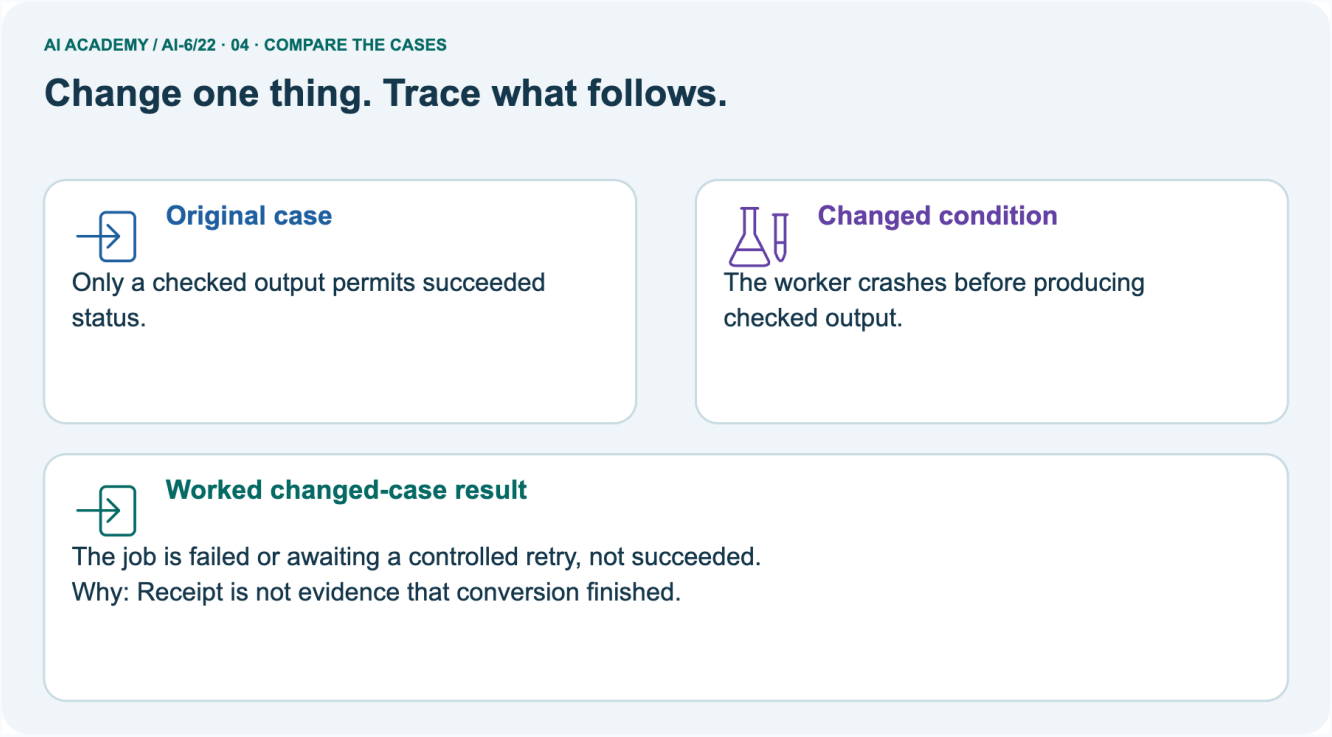
Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Asynchronous work separates submission from completion. Honest status and a job identifier let users track that difference.

Predict the pre-output crash

- Change one thing: The worker crashes before producing checked **output**. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain delivery limits and status evidence

- **Queue** delivery and duplicate handling need their own policies; this sequence does not guarantee completion.

- Read the labelled picture: Explain what the **queue**d J12 submission response proves and what it does not. Which event permits succeeded status, and what status is honest after a worker crash?

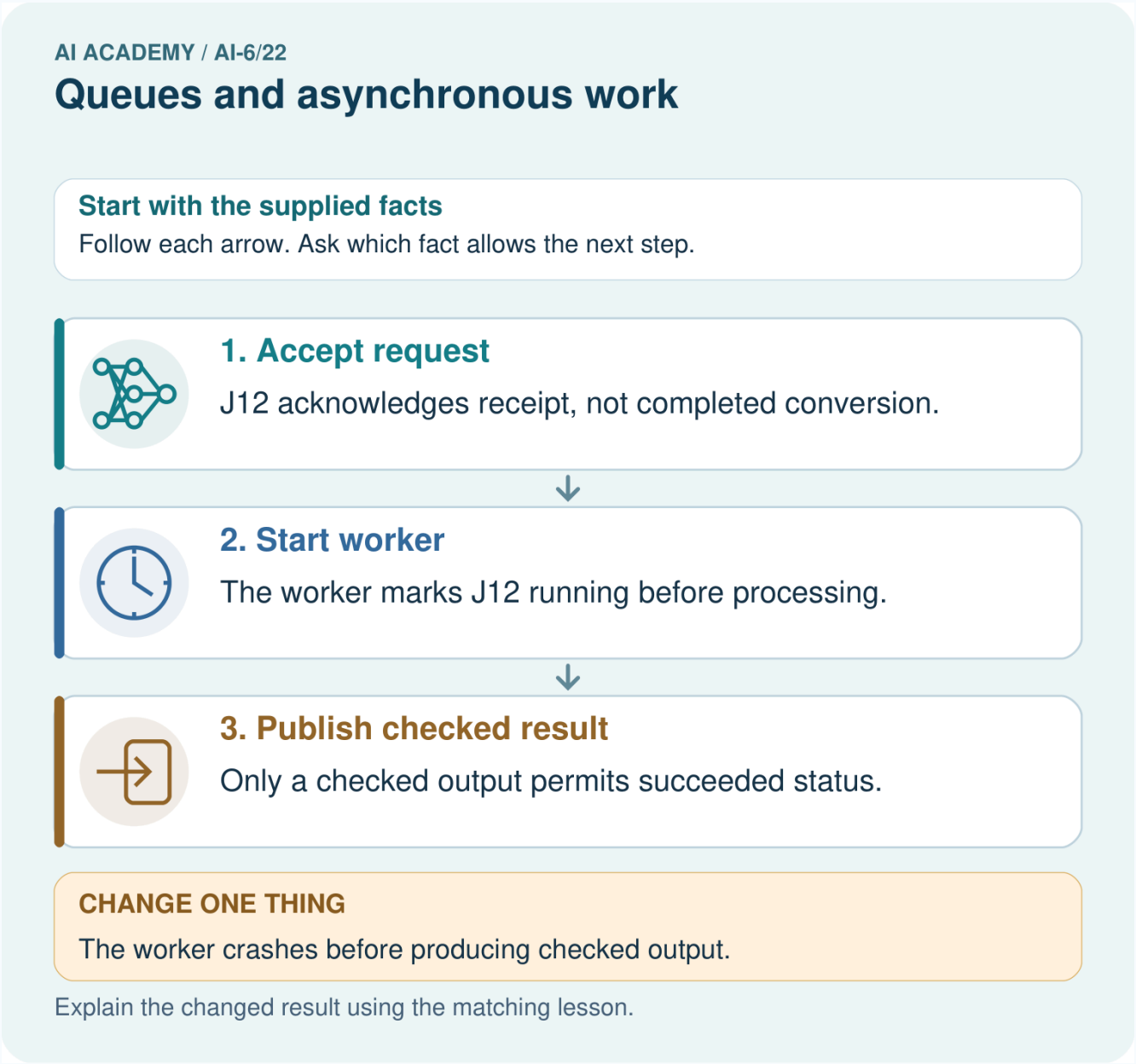
▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Queues and asynchronous work

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Audit evidence before counting completions or retries

> Delivery and effect counts answer different questions.

Core queue qualification and failure analysis

Queue ordering and exactly-once properties must be declared, not assumed from a simple mock.

A useful check asks what would make the reasoning fail. Distinguish a failure to execute the declared procedure from a weakness in the procedure itself.

The first may require a correction; the second requires an explicitly revised method and new **evidence**. Preserve unsuccessful cases so an improved result cannot hide the original problem.

Complete state, conflict and lost-history probes

Probe	Expected interpretation
Accepted job has only a receipt	Pending/queued; no success claim
Committed J delivered again with same payload	Return prior checked result, acknowledge, add no effect
Same ID with a different payload	Reject conflict
Saved status is running or absent	Reconcile; do not blindly replay an uncertain effect
Worker crash observed before checked output	Record failure or use a separately declared controlled retry
Crash not yet observed	May still appear running; no evidence of success
Completion history lost	Memory-only model can repeat effect; durability not established

Probe	Expected interpretation
Two workers run concurrently	Outside sequential mock; atomicity/locking/provider policy required
Several deliveries	Not necessarily several effects, and not proof of exactly-once delivery

Guided repair and explanation

- A report omits this qualification: **Queue** ordering and exactly-once properties must be declared, not assumed from a simple mock. Explain how omitting it could change a reader's interpretation, and write an accurate replacement.

Guided: restate the worked result, then name the assumption or limitation that prevents a broader conclusion.

State one repair and the checks you would rerun.

▼ Earlier wording, alternatives and picture transcripts

Locate the earliest unsupported step

Practise honest status reporting

> Name receipt, checked completion and uncertainty separately.

Current explanation and guided check

- Explain the weekly **mechanism** in your own words. Use the worked scenario to name its **input**, decision rule and **output**.

> **Guided: restate the worked result, then name the assumption or limitation that prevents a broader conclusion.**

> **Explain your prediction to a partner before comparing results.**

Supported W2 trace

Use workbook W2 for the connected example "**Queue**s and asynchronous work". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Individual scope check

- Use the worked case for **Queue**s and asynchronous work. Proposed explanation: “The worked result establishes every future case”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Picture response

- Explain what the **queue**d J12 submission response proves and what it does not. Which event permits succeeded status, and what status is honest after a worker crash?

> **My explanation using a pictured step and the given facts**
>
> _____
>
> _____
>
> _____
>
> _____

Reserve the selected three-job case

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace the worked case

Check your reasoning

Explain the picture

Run the complete redelivery and status-evidence model

> Compare preserved completion evidence with deliberately lost history.

Current crash-after-commit investigation

Worker commits job J then crashes before **queue acknowledgement**. J is delivered again. Show the intermediate steps and give the conclusion with its scope.

Runnable sequential mock

```
"""Sequential in-memory queue model; no durable store, broker or real effects."""
from copy import deepcopy

class MockWorker:
    def __init__(self, saved=None):
        self.jobs=deepcopy(saved or {})
        self.effects=[]
        self.deliveries=0
        self.acks=[]
    def accept(self,job_id,payload):
        if type(job_id) is not str or not job_id or type(payload) is not dict:
            raise ValueError('nonempty job ID and dictionary payload required')
        if job_id in self.jobs:
            if self.jobs[job_id]['payload']!=payload:raise ValueError('same ID has
conflicting payload')
        else:self.jobs[job_id]=
{'payload':deepcopy(payload),'state':'pending','result':None}
        return {'job_id':job_id,'state':self.jobs[job_id]['state']}
    def deliver(self,job_id,*,crash_after_commit=False):
        self.deliveries+=1
        job=self.jobs.get(job_id)
        if job is None:return {'status':'review','reason':'unknown ID; reconcile first'}
        if job['state']=='completed':
            self.acks.append(job_id)
            return {'status':'return_existing','result':deepcopy(job['result'])}
        if job['state']!='pending':return {'status':'review','reason':'not a new pending
job'}
        job['state']='running'
        # One indivisible classroom step represents a checked result plus recorded commit.
        # Real effect/status atomicity is an assumption, not implemented by this
dictionary.
        result={'artifact':'checked fictional output for '+job_id}
```

```
        self.effects.append(job_id)
        job.update(state='completed',result=result)
        if crash_after_commit:return {'status':'simulated_crash_before_ack'}
        self.acks.append(job_id)
        return {'status':'completed','result':deepcopy(result)}

worker=MockWorker();payload={'task':'fictional conversion'}
accepted=worker.accept('J',payload)
assert accepted['state']=='pending' and worker.effects==[]
first=worker.deliver('J',crash_after_commit=True)
assert worker.jobs['J']['state']=='completed' and worker.acks==[]
# Copy a declared saved snapshot into another worker: no persistence is actually tested.
saved=deepcopy(worker.jobs);recovered=MockWorker(saved)
second=recovered.deliver('J')
assert second['status']=='return_existing' and recovered.effects==[] and recovered.acks==
['J']
assert worker.effects==['J'] and worker.deliveries+recovered.deliveries==2
print('Main accepted:',accepted,'first delivery:',first,'redelivery:',second)
print('Two modeled deliveries; one original mock effect; redelivery adds zero effects.')
unknown=recovered.deliver('unknown')
assert unknown['status']=='review' and recovered.effects==[]
try:recovered.accept('J',{'task':'different'})
except ValueError:pass
else:raise AssertionError('conflicting ID accepted')
# A running snapshot cannot establish whether an external effect committed.
uncertain=MockWorker({'U':{'payload':{},'state':'running','result':None}})
assert uncertain.deliver('U')['status']=='review' and uncertain.effects==[]
# Losing all history makes prior completion unrecognizable: do not call this durable
dedup.
lost=MockWorker();lost.accept('J',payload);lost.deliver('J')
assert len(worker.effects)+len(lost.effects)==2
# Separate pictured vocabulary; observed failure before checked output supplied
explicitly.
picture_history=[{'id':'J12','state':'queued'},{'id':'J12','state':'running'},
                 {'id':'J12','state':'failed','reason':'observed crash before checked
output'}]
assert all(item['state']!='succeeded' for item in picture_history)
print('Picture observed-failure history:',picture_history)
print('An unobserved crash may leave running/stale status; reconcile rather than invent
failure or success.')
print('Pending/queued and completed/succeeded are paired classroom terms, not extra states
in one machine.')
print('Sequential copied fixtures do not implement durability, atomic commits,
concurrency, broker ordering, retries or exactly-once delivery.')
```

Expected effect counts and model limits

1. Predict J's state and effect count at acceptance, after commit/crash and on redelivery. The main path has two modeled deliveries and one mock effect. A copied saved snapshot lets a new worker return the previous result with no new effect. Unknown IDs and running-only snapshots go to review; changed payload under J is rejected.

   A separate lost-history demonstration repeats the mock effect, exposing the memory-only limitation. The J12 picture records an observed pre-output failure. This synchronous program creates no real queue, durable record, conversion or external effect; its single commit step assumes the atomic relationship a real system would need to implement. Record "not run" for a paper trace.

Evidence ledger and reflection

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or "not run"	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep "not run" if you only traced the procedure.

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: report three accepted jobs honestly

> Count completed, failed and pending without combining them into success.

Complete selected independent task

Jobs A, B, C accepted; A completed, B failed, C pending. How many completed, and may success be claimed for all accepted?

First attempt and support record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Apply a new case independently

Attempt this without hints

Jobs A, B, C accepted; A completed, B failed, C pending. How many completed, and may success be claimed for all accepted?

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

MAIN LESSON · SECTION 11

Save job IDs, state evidence and separate counts

> Keep the commit and acknowledgement events visible.

Project task and artifact

Add a labelled **queue**s and asynchronous work **evidence** sheet to Grounded Learning Service. Show how you can explain the **mechanism** and terminology of **queue**s and asynchronous work.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Actual run or trace and limitations

For the Grounded Learning Service, retain an annotated trace and the changed-case result. Name the contract or **evidence** that makes the deciding step valid, so another team can review it.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Explain what completion evidence a receipt lacks

> Use the job ID to connect observations without inventing state changes.

Retrieve and explain

- Close your notes. Explain this goal in your own words: Explain the **mechanism** and terminology of **queue**s and asynchronous work

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use service vocabulary precisely**

REFERENCE · SECTION 5

Reference · Use service vocabulary precisely

> Connect each term to an input, operation, evidence source or outcome.

Vocabulary and your explanation

Word	Meaning
Queue	A store of work awaiting processing
Acknowledgement	A signal allowing delivery completion
Redelivery	Receiving a previously delivered job again

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

Explain,repair and extend

- Explain the weekly **mechanism** in your own words. Use the worked scenario to name its **input**, decision rule and **output**.

- A report omits this qualification: **Queue** ordering and exactly-once properties must be declared, not assumed from a simple mock. Explain how omitting it could change a reader's interpretation, and write an accurate replacement.

- For the worked scenario, propose one additional case that could reveal a failure, describe its expected outcome before running it, and explain what would need to remain fixed for a fair comparison.

  Do not reuse W4 as a new final test after receiving feedback.

Explain → apply → challenge

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Reference · Check the service reasoning and its limits

REFERENCE · SECTION 12

Reference · Check the service reasoning and its limits

> Use after practice; keep the selected independent case for its first attempt.

Current explanation and response

- Complete the supported workbook tasks before attempting W4 independently. Record any hints or help used. After feedback, use the teacher's fresh retry rather than copying the earlier answer.

  For a project, select a relevant new case, predict the outcome from the declared rule and preserve the result alongside any revision.

- For the worked scenario, propose one additional case that could reveal a failure, describe its expected outcome before running it, and explain what would need to remain fixed for a fair comparison.

  Do not reuse W4 as a new final test after receiving feedback.

Exit: propose one revealing change to the worked scenario. State the expected outcome and the evidence that would make you revise your explanation.

My limited claim and a useful next check

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

Complete trace and evidence

Complete evidence record

Queues and asynchronous work — evidence trace

INPUT AND SCOPE	CHECK OR CALCULATION	CONCLUSION AND LIMIT
Worker commits job J then crashes before queue acknowledgement. J is delivered again.	A queued job is pending, running, completed or failed. Acknowledge only after the declared durable completion step. A repeated completed ID returns its prior result; unknown status needs reconciliation.	Consult durable job status; return committed result and acknowledge without repeating the effect. Delivery count is not effect count.

The figure connects this chapter's worked input, declared procedure and result. Read the complete worked reasoning for qualifiers and intermediate steps; the abbreviated labels do not replace that reasoning.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Reference · The earlier route checklist

REFERENCE · SECTION 15

Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

Targets,route and first prediction

Start with this question: **Queue**d is not complete. A conversion service **queue**s job J12 rather than **process**ing it during submission.

By the end, I can	How I will show it
Explain the mechanism and terminology of queues and asynchronous work	Explain the deciding step, show a trace or test, and state one limit.
Apply the stated procedure to a new case with a traceable result	Explain the deciding step, show a trace or test, and state one limit.
Identify an assumption or limitation and design a revealing follow-up	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Extra practice · Predict,check and repair

REFERENCE · SECTION 17

Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

First supported question and optional hint

- **Queue**d is not complete. A conversion service **queue**s job J12 rather than **process**ing it during submission.
- Which labelled step matches this detail? Only a checked **output** permits succeeded status.
- 1. Start worker
- 2. Publish checked result
- 3. Accept request
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check after the first attempt**

After your attempt, compare with the supported explanation: Publish checked result Only a checked **output** permits succeeded status. Asynchronous work separates submission from completion.

Honest status and a job identifier let users track that difference.

Changed-case question and optional hint

- Changed-case practice: The worker crashes before producing checked **output**. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. The job is failed or awaiting a controlled retry, not succeeded.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: The job is failed or awaiting a controlled retry, not succeeded. Receipt is not **evidence** that conversion finished.

Repair and retain the record

Review Caching and stale information (ai-6/21). Goal: Explain the **mechanism** and terminology of caching and stale information. Short practice: Identify the version mismatch in the cached 09:00 answer.

What should the first version -5 request return, and what permits a later version -5 cache hit?. Return to this week and explain what you changed.

Review Sandboxing and resource limits (ai-6/19). Goal: Explain the **mechanism** and terminology of sandboxing and resource limits. Short practice: Check A against both resource limits and B against network isolation.

Why must A stop after three seconds even if its **output** remains only 2 MB?. Return to this week and explain what you changed.

Review Timeouts retries and cancellation (ai-6/16). Goal: Explain the **mechanism** and terminology of timeouts retries and cancellation.

Short practice: Calculate waiting time for a one-second timeout followed by a 0.4-second success. What changes if cancellation arrives after the first attempt but before the retry?.

Return to this week and explain what you changed.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Keep your completed work

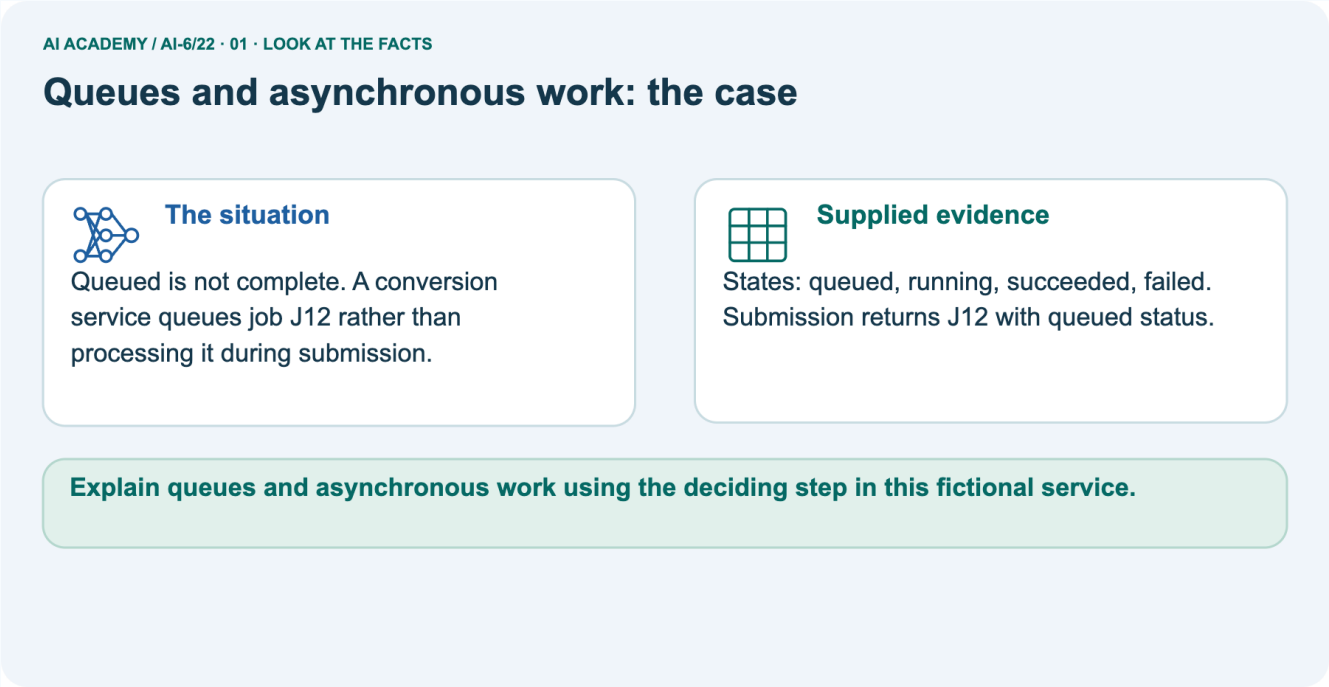
▼ **Reference · Six-picture concept journey**

REFERENCE · SECTION 18

Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

01. Queues and asynchronous work: the case

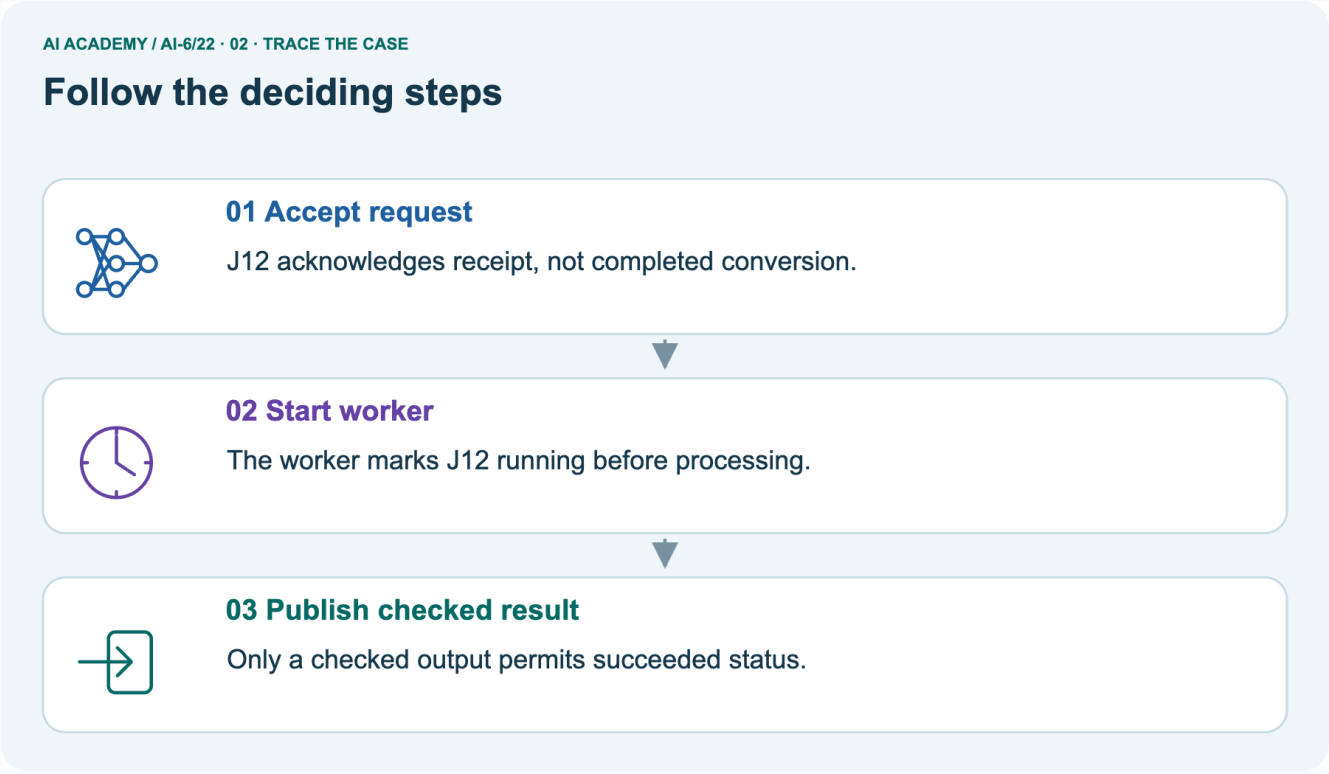


Follow the labelled picture with the matching explanation. Open the vector diagram

Queued is not complete. A conversion service **queue**s job J12 rather than **process**ing it during submission. Supplied facts: States: **queue**d, running, succeeded, failed. Submission returns J12 with **queue**d status.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

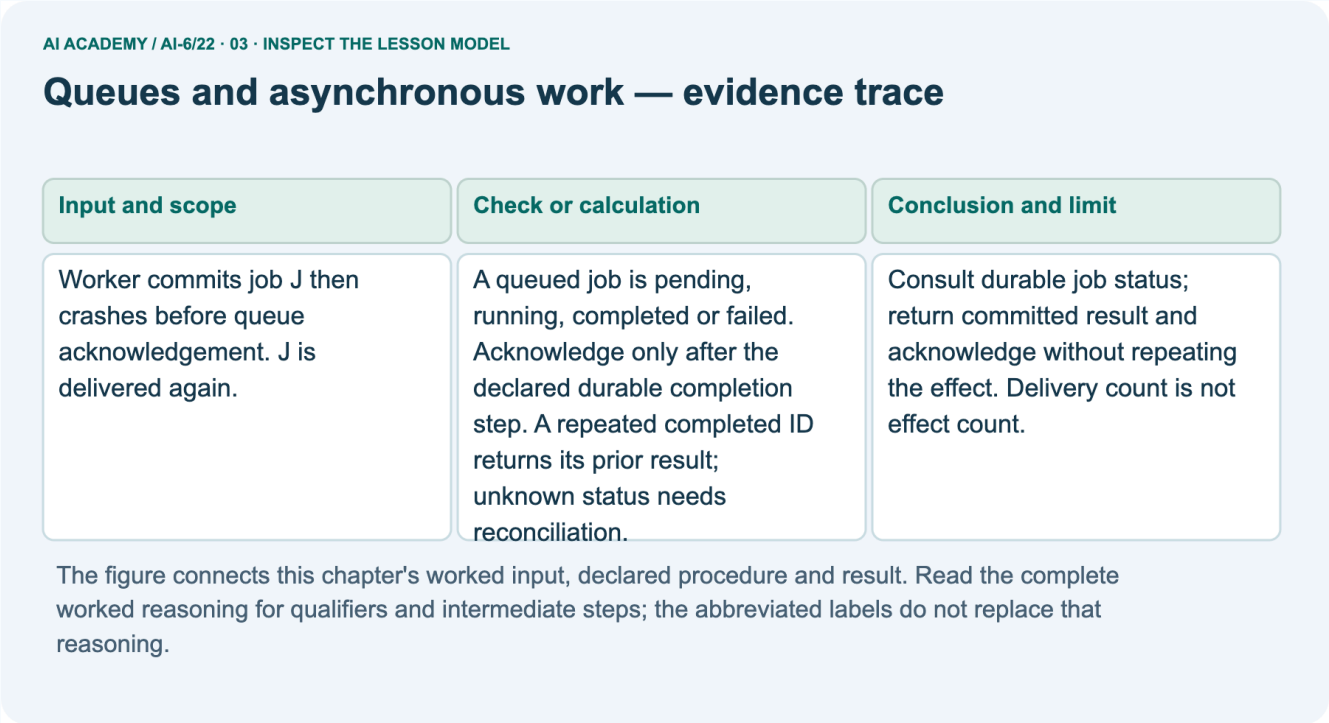


Follow the labelled picture with the matching explanation. Open the vector diagram

Accept request: J12 acknowledges receipt, not completed conversion. → Start worker: The worker marks J12 running before **process**ing. → Publish checked result: Only a checked **output** permits succeeded status.

Point to each numbered stage and explain which recorded **input** it uses.

03. Queues and asynchronous work — evidence trace



Follow the labelled picture with the matching explanation. Open the vector diagram

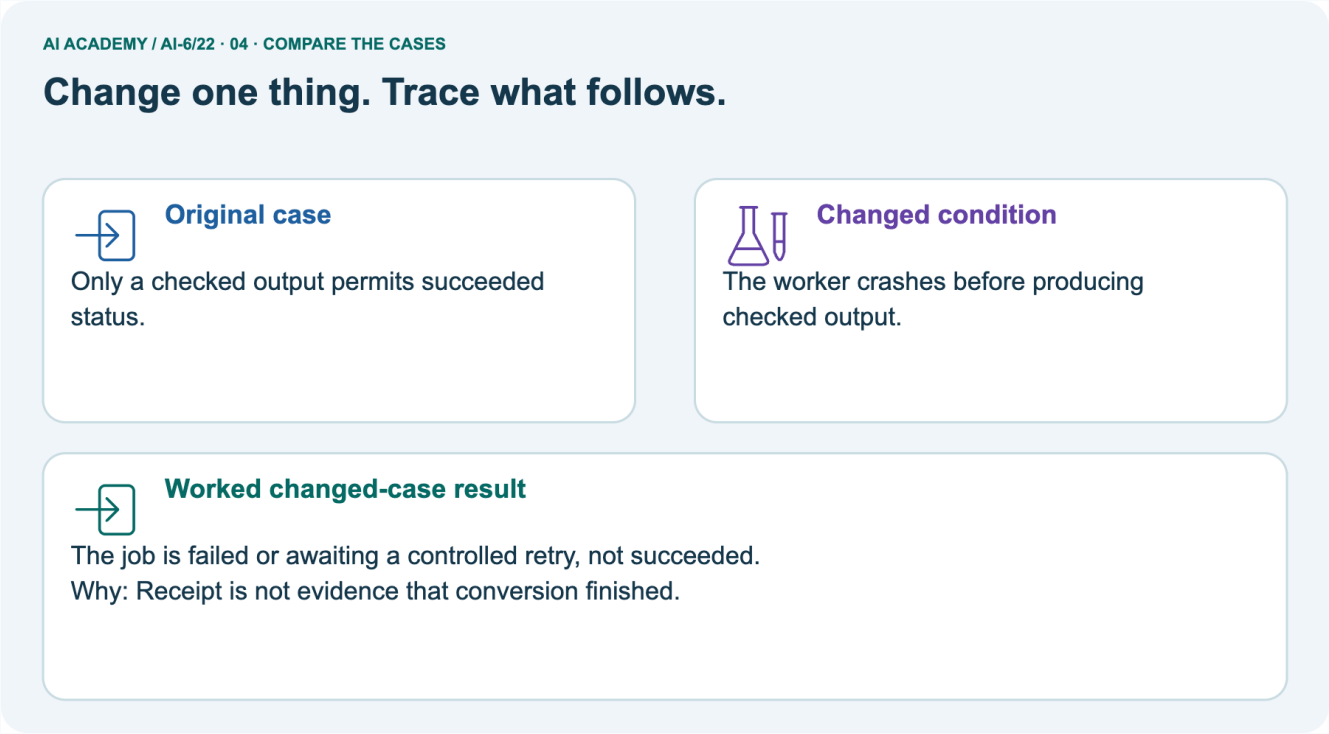
Queues and asynchronous work — **evidence** trace. Worker commits job J then crashes before **queue acknowledgement**. J is delivered again. | A **queue**d job is pending, running, completed or failed.

Acknowledge only after the declared durable completion step.

A repeated completed ID returns its prior result; unknown status needs reconciliation. | Consult durable job status; return committed result and acknowledge without repeating the effect. Delivery count is not effect count.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



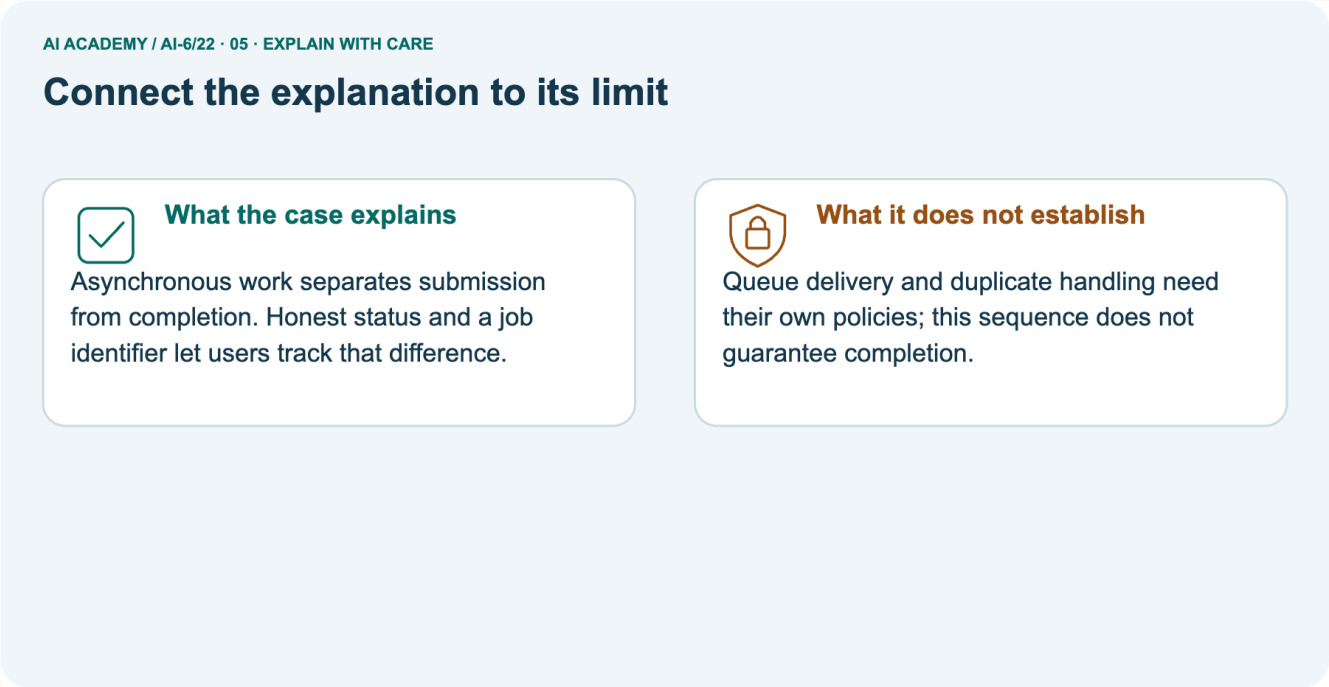
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Only a checked **output** permits succeeded status. Changed condition: The worker crashes before producing checked **output**. Expected result: The job is failed or awaiting a controlled retry, not succeeded.

Reason: Receipt is not **evidence** that conversion finished.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



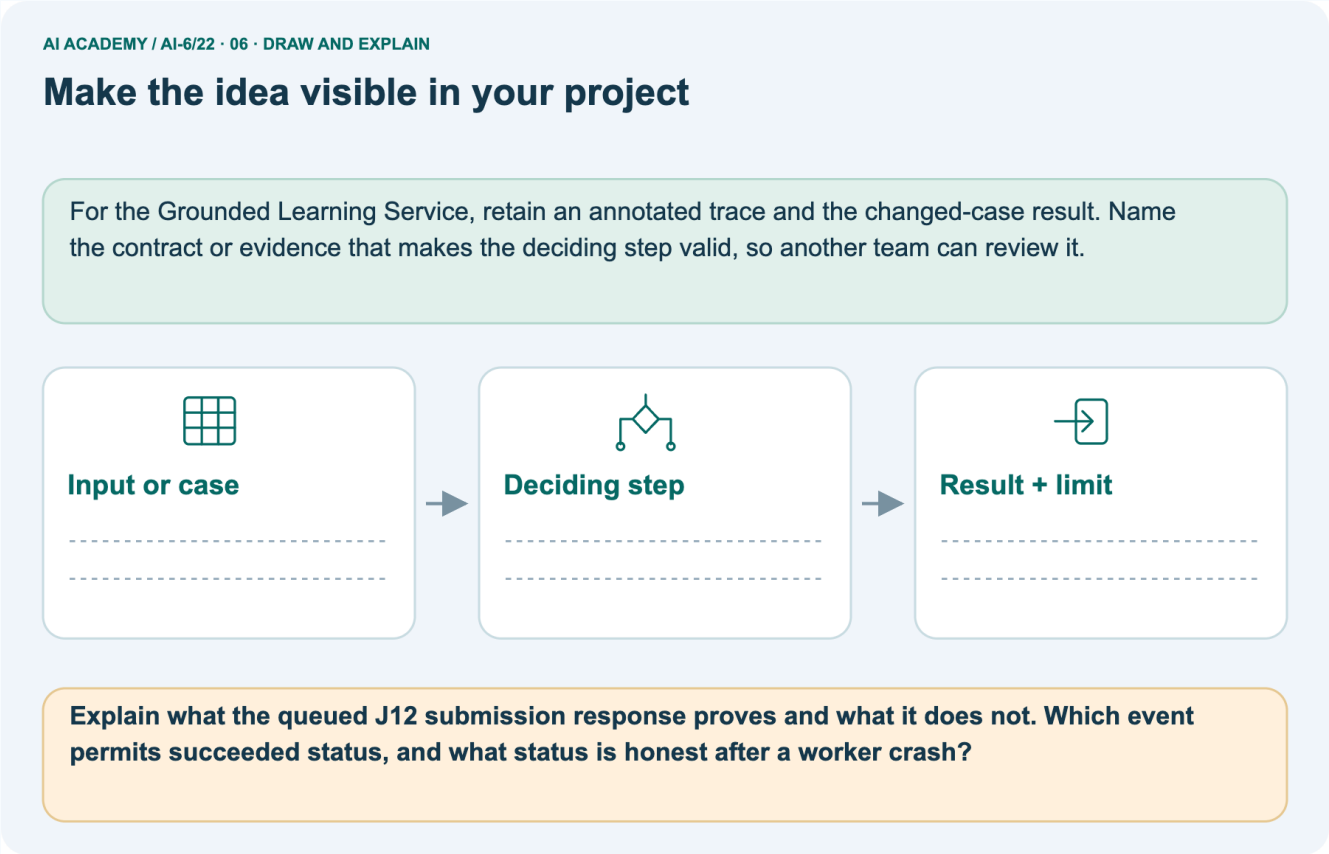
Follow the labelled picture with the matching explanation. Open the vector diagram

Asynchronous work separates submission from completion. Honest status and a job identifier let users track that difference.

Limit: **Queue** delivery and duplicate handling need their own policies; this sequence does not guarantee completion.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For the Grounded Learning Service, retain an annotated trace and the changed-case result. Name the contract or **evidence** that makes the deciding step valid, so another team can review it.

Draw your **input**, deciding step, result and limit. Supported question: Explain what the **queue**d J12 submission response proves and what it does not.

Which event permits succeeded status, and what status is honest after a worker crash?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Queues and asynchronous work: the case

02. Follow the deciding steps

03. Queues and asynchronous work — evidence trace

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.